

PHASE 2: REQUIREMENT ANALYSIS

TECHNOLOGY STACK

Date	28 February 2026
Team ID	LTVIP2026TMIDS76950
Team Size	4
Team Leader	Suresh Kumar V
Team Members	Vemanaboina Lakshmi Prasanna Chengalaraya Akhil Prasad Chittipothula
Project Name	HouseHunt: Finding Your Perfect Rental Home

1. TECHNOLOGY STACK OVERVIEW

Selected Stack: MERN

- **M** - MongoDB (Database)
- **E** - Express.js (Backend Framework)
- **R** - React.js (Frontend Library)
- **N** - Node.js (Runtime Environment)

2. FRONTEND TECHNOLOGIES

2.1 React.js (v18.2.0)

Purpose: UI Development

Why React? Component-based architecture enables reusability
Virtual DOM ensures high performance
Large ecosystem and community support
JSX syntax simplifies development
Unidirectional data flow easy to understand
Rich third-party library ecosystem
SEO-friendly with proper configuration

Alternatives Considered:

- Angular: Too heavy, steep learning curve
- Vue.js: Smaller ecosystem, less industry adoption

Use Cases in HouseHunt:

- Dashboard components for all user types

- Property cards and listings
 - Forms for registration, login, property addition
 - Modal dialogs for property details
 - Navigation and routing
-

2.2 React Router DOM (v6.15.0)

Purpose: Client-side routing

Features Used:

- Protected routes for authentication
- Nested routing for dashboards
- Dynamic routing for property details
- Programmatic navigation

Implementation:

Routes:

/ - Home page

/login - Login page

/register - Registration page

/adminhome - Admin dashboard (protected)

/ownerhome - Owner dashboard (protected)

/renterhome - Renter dashboard (protected)

2.3 Material-UI (MUI) v5.14.5

Purpose: UI Component Library

Components Used:

- Tables for data display
- Tabs for dashboard navigation
- Buttons and form controls
- Typography components
- Box and Grid layout
- Modal dialogs

Advantages:

- Professional design out-of-the-box
 - Responsive components
 - Accessibility built-in
 - Consistent design system
 - Extensive documentation
-

2.4 React Bootstrap (v2.8.0)

Purpose: Additional UI Components

Components Used:

- Navbar for navigation
- Cards for property display
- Forms for input collection
- Carousel for image galleries
- Modal for property details

Why Bootstrap + MUI?

- Bootstrap for layout and structure
 - MUI for data-heavy interfaces
 - Combined strengths of both libraries
-

2.5 Ant Design (v5.8.3)

Purpose: Message Notifications

Features:

- Success messages
- Error notifications
- Warning alerts
- Toast notifications

Advantage:

- Simple, elegant notification system
 - Non-intrusive user feedback
-

2.6 Axios (v1.4.0)

Purpose: HTTP Client

Why Axios over Fetch? Automatic JSON transformation

Request/response interceptors

Better error handling

Request cancellation

CSRF protection

Progress tracking for uploads

Usage:

- API calls to backend
- File uploads (property images)
- Token-based authentication headers

3. BACKEND TECHNOLOGIES

3.1 Node.js

Purpose: JavaScript Runtime

Why Node.js? JavaScript everywhere (frontend + backend)

Non-blocking I/O for high concurrency

npm ecosystem (1M+ packages)

Fast execution (V8 engine)

Excellent for RESTful APIs

Real-time capabilities

Active community support

Version: Latest LTS (Long Term Support)

3.2 Express.js (v4.18.2)

Purpose: Web Application Framework

Why Express? Minimal and flexible

Robust routing

Middleware support

HTTP utility methods

Template engine integration

Easy to learn

Industry standard

Features Used:

- RESTful API routing
- Middleware for authentication
- Error handling middleware

- Static file serving
- CORS configuration
- Body parsing

API Structure:

/api/user/* - User routes (registration, login, bookings)

/api/owner/* - Owner routes (property CRUD)

/api/admin/* - Admin routes (user management)

3.3 MongoDB

Purpose: NoSQL Database

Why MongoDB? Flexible schema (easy to modify)

JSON-like documents (natural fit for JavaScript)

Horizontal scalability

High performance

Rich query language

Aggregation framework

Cloud-ready (MongoDB Atlas)

Alternatives Considered:

- PostgreSQL: Too rigid for rapidly changing requirements
- MySQL: Schema changes difficult
- Firebase: Vendor lock-in concerns

Collections:

- users: User accounts (Admin, Owner, Renter)
 - properties: Property listings
 - bookings: Booking records
-

3.4 Mongoose (v7.4.3)

Purpose: MongoDB ODM

Features:

- Schema definition and validation
- Middleware (pre/post hooks)
- Query building
- Virtual properties

- Relationships (populate)
- Validation rules

Advantages:

- Structure for schema-less MongoDB
 - Built-in validation
 - Type casting
 - Query helpers
 - Cleaner code
-

4. AUTHENTICATION & SECURITY

4.1 JSON Web Tokens (JWT) v9.0.1

Purpose: Stateless Authentication

Why JWT? Stateless (no server-side session storage)

Scalable

Self-contained

Cross-domain compatible

Mobile-friendly

Token Structure:

Header.Payload.Signature

Payload Contains:

- User ID
- Issued at timestamp
- Expiration (24 hours)

Security:

- Secret key stored in environment variables
 - HTTPS transmission (production)
 - Token validation on every protected route
-

4.2 Bcryptjs (v2.4.3)

Purpose: Password Hashing

Features:

- One-way encryption

- Salt generation (10 rounds)
- Rainbow table attack prevention
- Adaptive hashing

Why Bcrypt? Industry standard

Proven security

Computationally expensive (prevents brute force)

Automatic salting

Never Store:

- Plain text passwords
 - Reversibly encrypted passwords
 - Weak hashes (MD5, SHA1)
-

5. FILE HANDLING

5.1 Multer (v1.4.5)

Purpose: File Upload Middleware

Features:

- Multipart form data handling
- File filtering (accept only images)
- File size limits
- Custom storage configuration
- Multiple file uploads

Configuration:

Storage: Disk storage

Destination: ./uploads/

Filename: Original filename

Accept: image/*

Max files: 10 per property

Security:

- File type validation
- File size limits
- Virus scanning (production)
- Secure file paths

6. DEVELOPMENT TOOLS

6.1 Nodemon (v3.0.1)

Purpose: Development Server

Features:

- Auto-restart on file changes
- Watch specific files/folders
- Delay restart
- Ignore patterns

Benefit:

- Faster development workflow
 - No manual server restarts
-

6.2 Git & GitHub

Purpose: Version Control

Usage:

- Code versioning
 - Collaboration
 - Backup
 - Documentation hosting
 - Issue tracking
 - Project deployment
-

6.3 VS Code

Purpose: IDE

Extensions Used:

- ESLint
- Prettier
- ES7 React snippets
- Thunder Client (API testing)
- GitLens

7. ADDITIONAL LIBRARIES

Frontend

- **prop-types:** Component prop validation
- **classnames:** Conditional CSS classes

Backend

- **cors:** Cross-Origin Resource Sharing
- **dotenv:** Environment variables
- **validator:** Input validation

8. DEPLOYMENT STACK (FUTURE)

Frontend Deployment

Platform: Vercel / Netlify

- Free tier available
- Automatic deployments
- CDN integration
- HTTPS by default

Backend Deployment

Platform: Heroku / Railway

- Free tier for testing
- Easy deployment
- Add-ons for MongoDB
- Automatic scaling

Database Hosting

Platform: MongoDB Atlas

- Free tier (512MB)
- Cloud-hosted
- Automatic backups
- Global distribution

File Storage (Future)

Platform: AWS S3 / Cloudinary

- Image optimization
- CDN delivery
- Transformations
- Scalable storage

9. TECHNOLOGY JUSTIFICATION MATRIX

Technology	Reason	Alternative	Why Not Alternative?
React.js	Component reusability	Angular	Too complex
Node.js	JavaScript everywhere	Django	Different language
MongoDB	Flexible schema	PostgreSQL	Rigid schema
Express	Minimal framework	Fastify	Less mature
JWT	Stateless auth	Sessions	Not scalable
Bcrypt	Industry standard	Argon2	Less support
Multer	Express integration	Formidable	Less popular

10. DEVELOPMENT ENVIRONMENT

Local Setup

Operating System: Windows 10/11 or macOS or Linux

Node.js Version: v14+ LTS

npm Version: v6+

MongoDB: Local installation or Atlas

Browser: Chrome (latest)

Memory: 4GB RAM minimum

Storage: 2GB free space

Environment Variables (.env)

MONGO_DB=mongodb://localhost:27017/househunt

PORT=8001

JWT_KEY=your-secret-key-here

11. PACKAGE MANAGEMENT

npm (Node Package Manager)

Why npm over yarn?

- Default with Node.js
- Largest package registry
- Wide adoption
- Well-documented

Key Commands:

npm install # Install dependencies

npm start # Run application

npm run dev # Development mode

npm test # Run tests

12. CODE QUALITY TOOLS

ESLint

Purpose: Code linting

- Catch errors early
- Enforce coding standards
- Consistent code style

Prettier

Purpose: Code formatting

- Automatic formatting
- Consistent style
- Save time

13. API TESTING TOOLS

Thunder Client (VS Code)

Purpose: API testing during development

Postman (Optional)

Purpose: API documentation and testing

Features:

- Request collections
- Environment variables
- Automated testing

- Documentation generation
-

14. PERFORMANCE CONSIDERATIONS

Frontend Optimization

- Code splitting (React.lazy)
- Image optimization
- Minification
- Caching strategies
- Lazy loading

Backend Optimization

- Database indexing
 - Query optimization
 - Connection pooling
 - Caching (Redis - future)
 - Load balancing (future)
-

15. SECURITY MEASURES

Implemented: Password hashing (bcrypt)

JWT authentication

CORS configuration

Input validation

SQL injection prevention (Mongoose)

XSS protection (React)

Environment variables

Planned:

- Rate limiting
 - HTTPS enforcement
 - Security headers (helmet)
 - Input sanitization
 - CSRF tokens
-

16. SCALABILITY STRATEGY

Current (MVP):

- MongoDB Atlas free tier
- Heroku free tier
- Single server instance

Future Scaling:

- Database sharding
- Horizontal scaling
- Load balancers
- CDN for static files
- Caching layer
- Microservices architecture

17. TECHNOLOGY LEARNING CURVE

Technology Difficulty Learning Time Documentation Quality

React.js	Medium	2-3 weeks	Excellent
Node.js	Easy	1 week	Excellent
Express.js	Easy	3-5 days	Excellent
MongoDB	Easy	1 week	Excellent
Mongoose	Medium	1 week	Good
JWT	Easy	2-3 days	Good

Total Learning Time: 6-8 weeks for complete proficiency

18. COST ANALYSIS

Development Cost

- All technologies: **FREE** (Open source)
- IDE (VS Code): **FREE**
- MongoDB Atlas: **FREE** tier (512MB)
- GitHub: **FREE** for public repos
- Deployment: **FREE** tiers available

Total Development Cost: ₹0

Production Cost (Monthly)

- MongoDB Atlas: ₹0 (Free tier) or ₹500 (Starter)
 - Heroku: ₹0 (Free) or ₹500 (Hobby)
 - Vercel/Netlify: ₹0 (Free tier)
 - Domain: ₹500/year
 - **Estimated Monthly:** ₹0 (Free tiers) or ₹1000-2000 (Paid tiers)
-

19. TECHNOLOGY ADOPTION TRENDS

All selected technologies are widely adopted:

- **React:** 11M+ weekly downloads
- **Express:** 20M+ weekly downloads
- **Mongoose:** 2M+ weekly downloads
- **JWT:** Industry standard
- **MongoDB:** Top 5 databases worldwide

Future-Proof: All technologies have strong community support and active development.

20. CONCLUSION

The MERN stack was selected after careful evaluation of requirements, alternatives, and constraints. Key factors:

JavaScript Everywhere: Single language reduces context switching

Rapid Development: Rich ecosystem accelerates development

Scalability: Cloud-native technologies support growth

Cost-Effective: Free and open-source

Community Support: Large communities for problem-solving

Industry Standard: Highly employable skills

Modern Architecture: Suitable for current and future needs

This stack provides the perfect balance of performance, scalability, developer experience, and cost-effectiveness for HouseHunt.

Document Status: Complete

Date: 28 February 2026

Part of HouseHunt Documentation - Smart Bridge Virtual Internship